

Teoria LP modulo 1

domenica 7 dicembre 2025

12:05

DEF: una grammatica libera da contesto è una quadrupla (NT, T, R, S) dove:

- NT insieme finito di non terminali
- T insieme finito di terminali
- $S \in NT$ simbolo iniziale
- R insieme finito di regole della forma $V \rightarrow w$ con $V \in NT$, $w \in (T \cup NT)^*$

DEF: il linguaggio generato da una gramm. $G = (NT, T, S, R)$ è l'insieme
 $L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$

PROP: in generale esistono infinite grammatiche diverse che generano lo stesso linguaggio

DEF: data una gram. $G = (NT, T, S, R)$, un albero di derivazione (o di parsing) è un albero ordinato in cui:

- ogni nodo è etichettato con un simbolo $\in T \cup NT \cup \{\epsilon\}$
- la radice è etichettata con S
- ogni nodo interno è etichettato con un NT
- se un nodo n ha etichetta $A \in NT$ e suoi figli sono nell'ordine $m_1 \dots m_k$ con etichetta $x_1 \dots x_k \in T \cup NT$ allora $A \rightarrow x_1 \dots x_k \in R$
- ogni nodo foglia è etichettato su $T \cup \{\epsilon\}$ allora l'albero corrisponde ad una derivazione completa

TEO: una stringa $w \in T^*$ appartiene a $L(G)$ se, e solo se, ammette un albero di derivazione completo.

DEF: una grammatica libera è ambigua se $\exists w \in L(G)$ che ammette più alberi di derivazione

DEF: un linguaggio L è ambiguo se $\forall G \vdash L = L(G)$ G è ambigua

DEF: un sistema di transizione è una tripla $\langle \Gamma, T, \rightarrow \rangle$ dove:

- Γ è l'insieme di stati (possibilmente infinito)
- $T \subseteq \Gamma$ è l'insieme degli stati terminali
- $\rightarrow \subseteq \Gamma \times \Gamma$ è la relazione di transizione

DEF: un token è una coppia (nome, valore) dove il nome è un simbolo astratto che rappresenta una categoria sintattica, il valore invece è una sequenza di simboli del testo in ingresso.

Inoltre un pattern è la descrizione generale della forma dei valori di una classe di token (Eg: l'expr. regolare $(a|b)(a|b)^*$). Un lessema è una stringa istanza di un pattern.

DEF: fissato un alfabeto $A = \{a_1 \dots a_n\}$ definiamo le espressioni regolari su A con la seguente BNF: $r ::= \emptyset \mid \varepsilon \mid a \mid r \cdot r \mid r_1 r_2 \mid r^*$ con la precedenza che è $r^* > r \cdot r > r_1 r_2$

DEF: dato l'alfabeto A , definiamo la funzione $L: \text{EXP-REG} \rightarrow 2^{A^*}$ così:

- $L[\emptyset] = \emptyset$
- $L[\varepsilon] = \{\varepsilon\}$
- $L[a] = \{a\}$
- $L[r_1 \cdot r_2] = L[r_1] \cdot L[r_2]$
- $L[r_1 | r_2] = L[r_1] \cup L[r_2]$
- $L[r^*] = (L[r])^*$

DEF: un linguaggio $L \subseteq A^*$ è detto regolare sse $\exists \text{ exp.-reg } r \text{ t.c. } L = L[r]$

PROP: ogni linguaggio finito è regolare.

DEF: due espressioni regolari r, s sono equivalenti ($r \approx s$) sse $L[r] = L[s]$

DEF: un automa finito nondeterministico NFA è una quintupla $(\Sigma, Q, \delta, q_0, F)$

dove:

- Σ è un alfabeto finito di simboli in input

- Q è un insieme finito di stati

- $q_0 \in Q$ è lo stato iniziale

- $F \subseteq Q$ è l'insieme degli stati finali

- δ è la funzione di transizione con tipo $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$

dove 2^Q è l'insieme delle parti di Q e $\delta(q, a) = Q' \subseteq Q$

DEF: un NFA $N = (\Sigma, Q, \delta, q_0, F)$ accetta $w = a_0 a_1 \dots a_n$ se nel diagramma di transizione $\exists$ cammino da q_0 a uno stato in F nel quale la stringa che si ottiene concatenando le etichette degli archi è esattamente w .

DEF: un automa finito deterministico è una quintupla $(\Sigma, Q, \delta, q_0, F)$ dove Σ, Q, q_0, F come negli NFA e $\delta: Q \times \Sigma \rightarrow Q$ (quindi $\delta(q, a)$ è sempre un singoletto)

OSS: un DFA è un particolare tipo di NFA $t.c.$:

- $\forall q \in Q \quad \delta(q, \epsilon) = \emptyset$ quindi non ci sono transizioni ϵ
- $\forall a \in \Sigma, \forall q \in Q, \exists q' \in Q$ $t.c.$ $\delta(q, a) = \{q'\}$ l'insieme delle mosse possibili è sempre un singoletto

PROP: per ogni NFA N è possibile costruire un DFA M ad esso equivalente con la tecnica di "costruzione per sottoinsiemi": $L[N] = L[M]$

DEF: sia q uno stato di un NFA. La ϵ -closure di q è l'insieme di stati dell'NFA raggiungibili da q solo con mosse ϵ .

OSS: grazie alla ϵ -closure si può definire il linguaggio riconosciuto da un NFA N come segue:

- $\hat{\delta}: Q \times \Sigma^* \rightarrow 2^Q$
- $\hat{\delta}(q, \epsilon) = \epsilon\text{-closure}(q)$
- $\hat{\delta}(q, xa) = \epsilon\text{-closure}(P)$ con $P = \{p \in Q \mid \exists v \in \hat{\delta}(q, x), p \in \delta(v, a)\}$

Quindi $w \in L[N] \iff \exists p \in F$ $t.c.$ $p \in \hat{\delta}(q_0, w)$

TEO: data un exp. reg. s possiamo costruire un NFA $N[s]$ $t.c.$ $L[s] = L[N[s]]$

DEF: una grammatica libera è regolare se ogni produzione è della forma $V \rightarrow aW$ oppure $V \rightarrow a$ dove $V, W \in NT$, $a \in T$ e per S è ammessa $S \rightarrow \epsilon$.

TEO: data una grammatica regolare G si può costruire un NFA N_G equivalente, con $Q = NT \cup \{\epsilon\}$, $F = \{\epsilon\}$, $q_0 = S$ e δ definita come segue:

- $Z \in \delta(V, a) \iff V \rightarrow aZ \in R$
- $\epsilon \in \delta(V, a) \iff V \rightarrow a \in R$
- $\epsilon \in \delta(S, \epsilon) \iff S \rightarrow \epsilon \in R$

TEO: da un DFA M possiamo definire una grammatica regolare G $t.c.$ $L(G) = L[M]$:

sia $M = (\Sigma, Q, \delta, q_0, F)$ allora $G = (Q, \Sigma, R, q_0)$ è definita ponendo

- per NT gli stati di M
- per T l'alfabeto di M
- per simbolo iniziale lo stato q_0
- per produzioni R I. $\forall \delta(q_i, a) = q_j$ la produzione $q_i \rightarrow a q_j \in R$ (e se $q_j \in F$ anche $q_i \rightarrow a$)

II. se $q_0 \in F$, allora $q_0 \rightarrow \epsilon \in R$

TEO: il linguaggio definito da una grammatica regolare G è regolare: è quindi possibile costruire un exp.-reg. S t.c. $L(G) = L[S]$.

OSS: espressioni regolari, NFA, DFA e grammatiche regolari sono tutti formalismi equivalenti che generano/riconoscono/descrivono la classe dei ling. regolari.

DEF: due stati q_1 e q_2 di un DFA M sono equivalenti se $\forall x \in \Sigma^*$ vale $\hat{\delta}(q_1, x) \in F \iff \hat{\delta}(q_2, x) \in F$ cioè se $L[M, q_1] = L[M, q_2]$

TEO: dato un DFA $M = (\Sigma, Q, \delta, q_0, F)$ l'algoritmo di minimizzazione con la tab. a scala termina e due stat. p, q sono equivalenti sse la casella (p, q) non è marcata

TEO: dato un DFA $M = (\Sigma, Q, \delta, q_0, F)$, l'automa $M_{min} = (\Sigma, Q_{min}, \delta_{min}, [q_0], F_{min})$ riconosce lo stesso linguaggio di M ($L[M] = L[M_{min}]$) ed ha il minimo numero di stati tra tutti gli automi deterministici per questo linguaggio.

TEO: Pumping Lemma: se L è regolare, allora $\exists N > 0$ t.c. $\forall z \in L$ con $|z| \geq N$, $\exists u, v, w$ t.c.

$$I) z = uvw \quad III) |v| \geq 1$$

$$II) |uv| \leq N \quad IV) \forall k \geq 0 \quad uv^k w \in L$$

ed N è al più il numero degli stat. del DFA minimo che accetta L .

PROP: la classe dei linguaggi regolari è chiusa per: unione, concatenazione, stella di Kleene, complementazione e intersezione.

DEF: una derivazione in cui, a partire da S , viene sempre riscritto il NT più a sinistra (destra) è detta derivazione canonica sinistra (destra) o left-most (rightmost).

DEF: un automa a pila non deterministico (PDA) è una sptupla $(\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ dove Σ, Q, q_0, F come negli automi visti in precedenza, mentre:

- Γ è un insieme finito di simboli sulla pila

- $\perp \in \Gamma$ è il simbolo iniziale sulla pila

- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \longrightarrow 2^{Q \times \Gamma^*}$

OSS: i PDA possono riconoscere un linguaggio per stato finale: $L[N] = \{w \in \Sigma^* \mid (q_0, w, \perp) \vdash_n^* (q, \varepsilon, \alpha) \text{ con } q \in F\}$ o per pila vuota: $P[N] = \{w \in \Sigma^* \mid (q_0, w, \perp) \vdash_n^* (q, \varepsilon, \varepsilon)\}$. Notare che spesso $P[N] \neq L[N]$.

TEO:
• se $L = P[N]$, possiamo costruire N' t.c. $L = L[N']$
• se $L = L[N]$, possiamo costruire N' t.c. $L = P[N']$

TEO: un linguaggio L è libero sse è accettato da un PDA.

TEO: i linguaggi liberi sono chiusi per unione, concatenazione e ripetizione.

TEO: l'intersezione $L_1 \cap L_2$ di un linguaggio libero L_1 e di uno regolare L_2 è un linguaggio libero.

OSS:
• se $L_1 \cap L_2$ non è regolare, ma L_1 è regolare allora L_2 è non regolare.
• se $L_1 \cap L_2$ non è libero, ma L_1 è regolare allora L_2 non è libero.

TEO: Pumping Theorem: se L è libero, allora $\exists N > 0$ t.c. $\forall z \in L, |z| \geq N \exists uvwx$ t.c.
I) $z = uvwx$ II) $|vx| \geq 1$
III) $|vwx| \leq N$ IV) $\forall k > 0 \quad uv^kwx^ky \in L$

DEF: un PDA $N = (\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ è deterministico (DPDA) se, e solo se,
I) $\forall q \in Q, \forall z \in \Gamma$ se $\delta(q, \varepsilon, z) \neq \emptyset \Rightarrow \delta(q, a, z) = \emptyset \quad \forall a \in \Sigma$ (se c'è una transizione ε allora non vi sono transizioni di altro tipo dallo stesso stato).
II) $\forall q \in Q, \forall z \in \Gamma \quad \forall a \in \Sigma \cup \{\varepsilon\}$ vale che $|\delta(q, a, z)| \leq 1$ (al più una mossa per ogni input "a").

DEF: un linguaggio è libero deterministico se è accettato per stato finale ($L = L[N]$) da un DPDA N .

TEO: la classe dei linguaggi liberi deterministici è inclusa propriamente nella classe dei linguaggi liberi.

PROP: se L è regolare, allora $\exists$ DPDA N t.c. $L[N] = L$ (riconoscimento per stato finale).

OSS: un linguaggio L libero deterministico è riconosciuto per pila vuota da un DPDA se, e solo se, L gode della prefix property: $\exists x, y \in L$ t.c. x è prefisso di y .

Osserviamo che se L è libero deterministico, allora $L\$ = \{w\$ \mid w \in L\}$ gode di pp.

PROP: se L è libero deterministico allora è generabile da una grammatica libera non ambigua. Segue da ciò che i linguaggi liberi del. non sono ambigui.

OSS: i linguaggi liberi deterministici sono chiusi per complementazione, cioè se $\exists \text{DPDA } N \vdash L = L[N]$, allora $\exists \text{DPDA } N' \vdash \bar{L} = L[N']$ dove $\bar{L} = \Sigma^* \setminus L$.

Non sono però chiusi per unione ed intersezione.

DEF: data $G = (NT, T, S, R)$ libera, un top-down parser è un PDA $M =$

$= (T, \{q\}, T \cup NT, \delta, q, S, \emptyset)$ che riconosce per pila vuota con funzione δ

definita così:

- $(q, \beta) \in \delta(q, \epsilon, A)$ se $A \rightarrow \beta \in R$ operazione di espansione;
- $(q, \epsilon) \in \delta(q, a, a) \forall a \in T$ operazione di consumo o match;

tale che $L(G) = P[M]$. Ovviamente questo parser è non deterministico.

OSS: non tutte le grammatiche sono adatte al top-down parsing.

DEF: un parser bottom-up per $G = (NT, T, S, R)$ è un PDA M che riconosce $L(G) \$$ e

$M = (T, \{q\}, T \cup NT \cup \{Z\}, \delta, q, Z, \emptyset)$ dove la funzione di transizione δ :

- $(q, \alpha X) \in \delta(q, a, X) \forall a \in T \forall X \in T \cup NT \cup \{Z\}$ operazione di shift
- $(q, A) \in \delta(q, \epsilon, \alpha^R)$ se $A \rightarrow \alpha \in R$ operazione di reduce
- $(q, \epsilon) \in \delta(q, \$, SZ)$ operazione di accept

Anche in questo parser c'è del non-determinismo.

PROP: data la grammatica libera G , la grammatica G' ottenuta con l'algoritmo di eliminazione delle ϵ -produzioni riportato in seguito non ha ϵ -produzioni e $L(G') = L(G) \setminus \{\epsilon\}$.

Algoritmo: 1) calcoliamo i simboli annullabili: $N(G)$ così: $N_0 = \{A \mid A \in NT; A \rightarrow \epsilon \in R\}$ e $N_{i+1} = N_i \cup \{B \mid B \in NT; B \rightarrow C_1 \dots C_k \in R; C_1, \dots, C_k \in N_i\}$

2) $\forall \text{prod } A \rightarrow \alpha \in R, \alpha \neq \epsilon$ in cui occorrono $Z_1, \dots, Z_k \in N(G)$ metto in R' le prod $A \rightarrow \alpha'$ con α' ottenuto da α eliminando i possibili sottainsiemi di $Z_1, \dots, Z_k$ e se α' risulta ϵ non lo mettiamo.

PROP: data la grammatica libera G e sia G' la grammatica ottenuta dall'alg. di eliminazione delle produzioni unitarie, vale $L(G) = L(G')$.

Algoritmo: 1) calcolo $U(G)$: $U_0 = \{A, A \mid A \in NT\}$, $U_{i+1} = U_i \cup \{(A, C) \mid (A, B) \in U_i; B \rightarrow C \in R\}$

2) se $G = (NT, T, R, S)$, $G' = (NT, T, R', S)$ dove $\forall (A, B) \in U(G)$, R' contiene tutte le prod $A \rightarrow \alpha$ dove $B \rightarrow \alpha \in R$ non è unitaria

DEF: un simbolo $X \in T \cup NT$ è • un generatore sse $\exists w \in T^*$ con $X \Rightarrow^* w$
• raggiungibile sse $S \Rightarrow^* \alpha X \beta$ per $\alpha, \beta \in (T \cup NT)^*$
• utile se è sia generatore che raggiungibile.

PROP: una grammatica G da cui eliminando i simboli inutili si ottiene G' tale per cui $L(G) = L(G')$. Per eliminare i simboli inutili prima elimino tutti i non generatori e le produzioni che li usano, poi elimino tutti i simboli non raggiungibili.

Nota: • $G_0 = T$, $G_{i+1} = G_i \cup \{B \in NT \mid B \rightarrow C_1 \dots C_k \in R; C_1, \dots, C_k \in G_i\}$
• $R_0 = \{S\}$, $R_{i+1} = R_i \cup \bigcup_{\substack{B \in R: \\ B \rightarrow x_1 \dots x_k \in R}} \{x_1, \dots, x_k\}$

OSS: per semplificare una grammatica G elimino prima le ϵ -prod, poi le produzioni unitarie e in fine i simboli inutili.

DEF: una grammatica è nella Forma Normale di Chomsky se tutte le sue produzioni sono $A \rightarrow BC$ o $A \rightarrow a$.

DEF: una grammatica è nella Forma Normale di Greibach se tutte le sue produzioni sono $A \rightarrow aBC$ o $A \rightarrow aB$ o $A \rightarrow a$

OSS: se G è in una delle due forme elencate prima allora è priva di ϵ -produzioni e produzioni unitarie.

Inoltre ogni G libera può essere trasformata in una grammatica equivalente in una delle due forme.

DEF: data una grammatica libera G e $\alpha \in (T \cup NT)^*$ diciamo che $\text{first}(\alpha)$ è l'insieme dei terminali che possono stare in prima posizione in una stringa che si deriva da α .

DEF: per G libera diciamo che il $\text{follow}(X)$ con $X \in NT$ è l'insieme dei terminali che possono comparire immediatamente a destra di X in una forma sentenziale.

$\$ \in \text{follow}(S)$ sempre

$\forall R$, se R è $A \rightarrow \dots X \beta$ allora $\text{follow}(X) = \text{follow}(X) \cup (\text{first}(\beta) \setminus \{\epsilon\})$;

se è $A \rightarrow \dots X$ o $A \rightarrow \dots X \beta$ con $\beta \rightarrow \epsilon$ allora $\text{follow}(X) = \text{follow}(X) \cup \text{follow}(A)$

DEF: una tabella di parsing $LL(1)$ è una matrice bidimensionale M con:

- per righe i non terminali
- per colonne i terminali e $\$$
- nella casella $(A, a): M[A, a]$ contiene le prod. che possono essere scelte dal parser avendo "a" come input mentre tenta di espandere A .

Se ogni casella ha al massimo una produzione allora il parser è deterministico.

DEF: una grammatica è $LL(1)$ sse ogni casella della tabella $LL(1)$ contiene al più una produzione.

TEO: G è $LL(1)$ sse $\forall$ coppia di produzioni distinte che hanno la stessa testa $A \rightarrow \alpha \mid \beta$

I) $\text{first}(\alpha) \cap \text{first}(\beta) = \emptyset$

II) se $\epsilon \in \text{first}(\alpha)$, allora $\text{Follow}(A) \cap \text{first}(\beta) = \emptyset$. (Simmetrico per $\epsilon \in \text{first}(\beta)$)

TEO: ogni linguaggio regolare è generabile da una grammatica G di classe $LL(1)$

DEF: $w \in \text{first}_k(\alpha)$ sse $\alpha \Rightarrow^* w\beta$, $w \in T^*$, $\beta \in (T \cup NT)^*$ e $|w| \leq k$,

oppure sse $\alpha \Rightarrow^* w$ con $w \in T^*$ e $|w| \leq k$

DEF: $w \in \text{follow}_k(A)$ sse $S \Rightarrow^* \alpha A w \beta$ con $|w| \leq k$, $w \in T^*$ e $\alpha, \beta \in (T \cup NT)^*$,
oppure sse $S \Rightarrow^* \alpha A w$ con $|w| \leq k$ e $w \in T^*$

DEF: una tabella di parsing $LL(k)$ è una matrice con i NT come righe, per colonne $\{w \in T^* \mid |w| \leq k\}$ (solo quelli necessari).

Per ogni produzione $A \rightarrow \alpha$, $M[A, w]$ contiene $A \rightarrow \alpha$ $\forall w \in \text{first}_k(\alpha)$ esatto e $\forall w \in \text{follow}_k(\alpha)$ se $\epsilon \in \text{first}_k(\alpha)$.

Se ogni casella ha al più una produzione e $\nexists w_1, w_2$ t.c. w_1 prefisso di w_2 con le corrispondenti caselle entrambe riempite nella stessa riga, allora G è $LL(k)$.

- TEO:
- una grammatica ricorsiva sinistra non è mai $LL(k)$.
 - una grammatica ambigua non è mai $LL(k)$.
 - se G è $LL(k)$ allora non è ambigua.
 - se G è $LL(k)$ allora $L(G)$ è libero deterministico.
 - certi linguaggi liberi deterministici non possono essere generati da nessuna grammatica $LL(k)$.

DEF: un parser shift-reduce nondeterministico è un PDA $M = (T, \{q\}, T \cup NT \cup \{\$, \epsilon\}, \delta, q, \epsilon)$ dove:

- $(q, aX) \in \delta(q, a, X) \quad \forall a \in T, \forall X \in \Gamma$ operazione di shift
- $(q, A) \in \delta(q, \epsilon, \alpha^e)$ se $A \rightarrow \alpha \in R$ operazione di reduce
- $(q, \epsilon) \in \delta(q, \$, \$)$ operazione di accept

Questo parser è non deterministico a causa dei conflitti shift-reduce e dei conflitti reduce-reduce.

DEF: un prefisso viabile è una sequenza di terminali e non terminali che può apparire sulla pila di un parser bottom-up per una computazione che accetta un input.

In pratica per risolvere un conflitto bisogna che la parte top della pila sia un prefisso di una parte dx di una produzione.

TEO: data G libera, i prefissi viabili di G costituiscono un linguaggio regolare che può essere anche descritto da un DFA.

DEF: un item $LR(0)$ è una produzione con indicata, usando un punto, una posizione della sua parte destra.

DEF: l'NFA dei prefissi viabili di $G = (NT, T, S, R)$ libera (con anche la produzione $S' \rightarrow S$) si ottiene così:

- $[S' \rightarrow \cdot S]$ è lo stato iniziale
- dallo stato $[A \rightarrow \alpha \cdot X \beta]$ c'è una transizione ad $[A \rightarrow \alpha X \cdot \beta]$ etichettata con $X \in T \cup NT$
- dallo stato $[A \rightarrow \alpha \cdot X \beta]$, per $X \in NT$ e per ogni produzione $X \rightarrow \gamma$, c'è una transizione verso $[A \rightarrow \alpha \cdot \gamma \beta]$

DEF: una tabella di parsing LR è una matrice che ha gli stati dell'automa canonico $LR(0)/LR(1)$ e $T \cup \{\$, \}$ come colonne. $M[s, x]$ contiene le azioni che un parser LR può compiere con x simbolo in input e s stato top nella pila degli stati.

Ovviamente se la tabella contiene più azioni in una stessa cella vuol dire che il parser non è deterministico.

Per una tabella $LR(0)$: $\forall$ stato s dell'automa canonico $LR(0)$:

- se $x \in T$ e $s \xrightarrow{x} t$ nell'automa allora $M[s, x] = \text{"shift"}$
- se $A \rightarrow \alpha \cdot \in S$ e $A \neq S'$ inserisci "reduce $A \rightarrow \alpha$ " in $M[s, x] \forall x \in T \cup \{\$, \}$
- se $S' \rightarrow S \cdot \in S$, inserisci in $M[s, \$]$ "accept"
- se $A \in NT$ e $s \xrightarrow{A} t$ nell'automa $LR(0)$, $M[s, A] = \text{"goto"}$

DEF: una grammatica è LR(0) se ogni casella nella tabella di parsing contiene al massimo un'azione.

OSS: se G ha produzioni ϵ allora è difficile che sia LR(0)

OSS: se L è libero deterministico e gode della prefix-property allora L è LR(0).

Quindi: se L è libero det e non è LR(0) allora non vale la prefix-property.

OSS: se L è LR(0) ed è finito allora gode della prefix-property. Quindi: se L è finito e non vale la pref.-prop. allora L non è LR(0).

DEF: una tabella di parsing SLR(1) è una matrice come le tabelle di parsing LR(0)/LR(1) riempita quasi analogamente eccetto che per le azioni di reduce: se $A \rightarrow \alpha \cdot \epsilon S$ e $A \neq S'$, inserisci "reduce $A \rightarrow \alpha$ " in $M[s, x] \forall x \in \text{follow}(A)$.

DEF: un item LR(1) è una coppia formata da un item LR(0) detto core (nucleo) e da un simbolo di lookahead $\in T \cup \{\$ \}$.

DEF: un NFA LR(1) ha:

- come stati: gli item LR(1) della grammatica aumentata (con $S' \rightarrow S$);
- $[S \rightarrow \cdot S', \$]$ è lo stato iniziale
- dallo stato $[A \rightarrow \alpha \cdot X \beta, a]$ c'è una transizione ad $[A \rightarrow \alpha X \cdot \beta, a]$ etichettata $X \in T \cup NT$
- dallo stato $[A \rightarrow \alpha \cdot X \beta, a]$, per $X \in NT$ e $\forall \text{prod } X \rightarrow \delta$ c'è una ϵ -transizione verso $[X \rightarrow \cdot \delta, b]$ $\forall b \in \text{first}(\beta a)$. L'automa canonico LR(1) è il DFA ottenuto da questo NFA con la $\text{clos}(I)$ e la $\text{goto}(I, X)$ oppure con la costruzione per sottoinsiemi.

DEF: una grammatica libera G è di classe LR(1) se ogni cella della tabella LR(1) contiene al più un elemento.

PROP: se G è LL(k), allora G non è ambigua e $L(G)$ è deterministico.

PROP: se G è LR(k), allora G non è ambigua e $L(G)$ è deterministico.

PROP: • $SLR(k) \subset LALR(k) \subset LR(k) \forall k \geq 1$

• $SLR(1) \subset SLR(2) \subset \dots \subset SLR(k)$

• $LALR(1) \subset LALR(2) \subset \dots \subset LALR(k)$

• $LR(0) \subset LR(1) \subset \dots \subset LR(k)$

PROP: $\bullet LL(k) \subset LR(k) \quad \forall k \geq 0$

$\bullet LL(k) \not\subset LR(k-1) \quad \forall k \geq 1$

DEF: un linguaggio L è di classe X se $\exists G$ di classe X t.c. $L = L(G)$

TEO: un linguaggio è libero det. sse è generato da una grammatica $LR(k)$ per qualche $k \geq 0$

TEO: un linguaggio è libero det. sse $L = L(G)$ con G grammatica $SLR(1)$

TEO: i linguaggi generati da grammatiche $LL(k)$ sono strettamente contenuti nei linguaggi generati da gramm. $SLR(k) \quad \forall k \geq 0$